

テーマ: 脆弱なプログラム

1. はじめに

脆弱なプログラムとは、悪意のある攻撃者によって不正に利用される可能性のあるプログラムのことです。セキュリティ上の欠陥があると、システムが乗っ取られたり、情報が漏えいする危険があります。レポートでは、1. vuln_overflow.c 2. vuln_format.c 3. vuln_cmdexec.c の3つの脆弱性について学習した内容をまとめます。

1. vuln_overflow

プログラムには、strcpy() と gets() 関数による境界チェックの欠如が原因で、メモリ破壊や任意コード実行の危険性があった。

発見された脆弱性

1.1 strcpy() の問題

場所: greet() 関数内, コピー先のバッファ (char buf[32]) に対して、入力長のチェックを行っていない。

影響: 32 バイトを超える入力を受け取ると、スタック領域を上書きし、プログラムがクラッシュまたは任意コード実行される危険がある。

1.2 gets() の問題

場所: main() 関数内, 入力長を制限せずに文字列を取得する。

影響: 大量のデータ入力により、バッファを超過し不正アクセスが発生する可能性。

```
/* vuln_overflow.c */
#include <stdio.h>
#include <string.h>

void greet(const char *name) {
    char buf[32];
    /* unsafe copy: no bounds check */
    snprintf(buf, sizeof(buf), "%s", name);
    printf("Hello, %s\n", buf);
}

int main(void) {
    char input[256];
    printf("Enter your name: ");
    if (fgets(input, sizeof(input), stdin) == NULL) {
        fprintf(stderr, "Error reading input\n");
        return 1;
    }
    input[strcspn(input, "\r\n")] = '\0';
    greet(input);
    return 0;
}
```

vuln_overflow.c

```
/* vuln_overflow.c */
#include <stdio.h>
#include <string.h>

void greet(char *name) {
    char buf[32];
    /* unsafe copy: no bounds check */
    strcpy(buf, name);
    printf("Hello, %s\n", buf);
}

int main(void) {
    char input[256];
    printf("Enter your name: ");
    /* unsafe input: no bounds check */
    gets(input);
    greet(input);
    return 0;
}
```

1.3. 修正内容

- strcpy() → snprintf() 修正前: strcpy(buf, name) 修正後: snprintf(buf, sizeof(buf), %s, name)
- gets() → fgets() 修正前: gets(input) 修正後: if (fgets(input, sizeof(input), stdin) == NULL) // エラー処理 (入力長を制限し、バッファオーバーフローを防止)

4. テスト結果

```
=== Overflow tests for fix_overflow ===
--- input #1 (len=5) ---
Enter your name: Hello, Alice
--- input #2 (len=32) ---
Enter your name: Hello, AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
--- input #3 (len=33) ---
Enter your name: Hello,BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBB
--- input #4 (len=100) ---
Enter your name: Hello, CCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCC
--- input #5 (len=1000) ---
Enter your name: Hello, DDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDD
```

テスト内容	入力	結果
正常入力	Alice	正常動作
32 文字入力	AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA	正常動作
33 文字入力	AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA	安全に切り捨て
100 文字入力	A×100	エラーなし
1000 文字入力	A×1000	クラッシュなし、正常終了

1.4 結論

修正版のプログラム (fix_overflow.c) では、入力サイズ制御を適切に実施することで、バッファオーバーフローの危険性を完全に排除した。
これにより、プログラムの安定性と安全性が大幅に向上した。

2. vuln_format.c

vuln_format.c に含まれる フォーマット文字列脆弱性 (Format String Vulnerability) について分析し、修正版 (fix_format.c) での改善内容とその効果を説明する。

この脆弱性は、ユーザー入力を printf() 関数に直接渡していることが原因で、メモリ情報漏洩や任意コード実行につながる危険がある。

2.1. 発見された脆弱性

場所: main() 関数内 (printf(buf);)

脆弱性タイプ: CWE-134 (Format String Vulnerability)

深刻度: 高

危険なコード:

printf(buf); // ユーザー入力をそのままフォーマット文字列として解釈

vuln_format.c

```
/* vuln_format.c */
#include <stdio.h>
#include <string.h>

int main(void) {
    char buf[128];
    printf("Enter a message: ");
    if (fgets(buf, sizeof(buf), stdin) == NULL) return 1;
    /* vulnerable: user input used as format string */
    printf(buf);
    printf("%n");
    return 0;
}
```

```
/* vuln_format.c - FIXED */
#include <stdio.h>
#include <string.h>

int main(void) {
    char buf[128];

    printf("Enter a message: ");

    if (fgets(buf, sizeof(buf), stdin) == NULL) {
        return 1;
    }

    /* Remove newline */
    buf[strcspn(buf, "\r\n")] = '\0';

    /* FIXED: Use format specifier to prevent format string attack */
    printf("%s\n", buf);

    return 0;
}
```

2.2 修正内容

- 修正前 printf(buf); 修正後 printf("%s\n", buf);
- buf[strcspn(buf, " / r /n")] = /0; fgets で読み込まれる可能性のある / n と / r を削除し、文字列の末尾に正しいヌル終端 ('¥0') を設定するための処理です

2.3 テスト結果

```
=== Format tests for fix_format ===  
--- input #1 (len=12) ---  
Enter a message: Hello, world  
--- input #2 (len=13) ---  
Enter a message: Normal%20text  
--- input #3 (len=8) ---  
Enter a message: %x %x %x  
--- input #4 (len=6) ---  
Enter a message: %%s%%  
--- input #5 (len=120) ---  
Enter a message: EEEEEEEEEEEEEEEEEEEEEEEEEEEEEEEEEEEEEEEEEEEEEEEEEEEEEEE
```

入力例	修正後の動作
Hello, world	Hello, world
%x %x %x	%x %x %x と表示
%s%s%	%s%s% と表示
A×128 文字	正常表示

2.4 結論

プログラムに存在した Format String 脆弱性 は、ユーザー入力をフォーマット文字列として誤って扱っていたことが原因である。

修正後のコードでは、`printf(%, buf)` を用いることで入力を単なる文字列として安全に出力できるようになった。

3. vuln_cmdexec.c

vuln_cmdexec.c に存在する command injection 脆弱性を分析し、修正

(fix_cmdexec.c) の内容と効果をまとめたものである。該当箇所は ping 実行部分で、ユーザー入力をそのままシェルに渡す実装になっていたため深刻なリスクがある。

3.1 発見された脆弱性

プログラムは、ユーザーから入力されたホスト名を用いて ping コマンドを実行する。しかし、入力検証が不十分なため、攻撃者が任意のシステムコマンドを実行できる。

```
sprintf(cmd, "ping -c 1 %s", host);
system(cmd); // ユーザー入力をシェルに渡している
```

vuln_cmdexec.c

```
/* vuln_cmdexec.c */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void) {
    char host[128];
    char cmd[256];

    printf("Enter host to ping: ");
    if (fgets(host, sizeof(host), stdin) == NULL) return 1;
    host[strcspn(host, "\n")] = '\0';

    /* vulnerable: building shell command by concatenation */
    snprintf(cmd, sizeof(cmd), "ping -c 1 %s", host);
    system(cmd); /* risk: shell interprets injected characters */
    return 0;
}
```

脆弱なコード

```
int main(void) {
    char host[128];
    char cmd[256];

    printf("Enter host to ping ");
    if (fgets(host, sizeof(host), stdin) ==
        NULL) return 1;
    host[strcspn(host, "\n")] = '\0';
```

脆弱性：入力検証なし

```
    snprintf(cmd, sizeof(cmd), "ping -c 1 %s,
host);
```

危険：shell を経由してコマンド実行

```
    system(cmd);
```

3.4 修正内容

- 危険：system()
system(ping -c 1 127.0.0.1; rm -rf /); → shell が rm -rf /を実行
- 安全：execl()
execl(/bin/ping, ping, -c, 1, 127.0.0.1; rm -rf /, NULL);
→ 127.0.0.1; rm -rf というホスト名で ping
- → shell を経由しないため、; が解釈されない

```

/* fix_cmdexec.c -- safe replacement for vuln_cmdexec.c */
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <errno.h>
#include <stdlib.h>

int valid_host(const char *h) {
    if (h == NULL || *h == '\0') return 0;
    size_t len = strlen(h);
    if (len == 0 || len >= 128) return 0;
    if (h[0] == '-' || h[0] == '.') return 0;
    if (h[len - 1] == '-' || h[len - 1] == '.') return 0;

    for (const char *p = h; *p; ++p) {
        unsigned char c = (unsigned char)*p;
        if (!isalnum(c) || c == '.' || c == '-') return 0;
    }
    return 1;
}

int main(void) {
    char host[128];

    printf("Enter host to ping: ");
    if (fgets(host, sizeof(host), stdin) == NULL) {
        fprintf(stderr, "Error reading input\n");
        return 1;
    }

    host[strcspn(host, "\r\n")] = '\0';

    if (!valid_host(host)) {
        fprintf(stderr, "Invalid host. Allowed characters: letters, digits, '.' and '-'.\n");
        return 1;
    }

    pid_t pid = fork();
    if (pid < 0) {
        perror("fork");
        return 1;
    } else if (pid == 0) {

```

1. 入力 が NULL または空文字でないことを確認。

長さを 1～127 に制限 (127 文字まで許可)。

先頭・末尾に - や . が来ないようにする簡易ルール。各文字が 英数字／ドット (.) ／ハイフン (-) のみ であることを確認 (ホワイトリスト)。ホワイトリスト方式により ;, &, \$, () などのシェルメタ文字を事前に排除する。

2. int main (void)

- fgets を利用してバッファサイズを明示的に制限しているためバッファオーバーフローを防止。
- strcspn(host, " \r\n") によって ' \r\n' を検出し、その位置にヌル終端 '\0' を設定して改行を削除している。これにより後続の文字列検証や execl の引数に不要な改行が混入しません。valid_host が false を返した場合は処理を中断し、危険な入力を実行させない。

```

if (!valid host(host)) {
    fprintf(stderr, "Invalid host. Allowed characters: letters, digits, '.' and '-'.\\n");
    return 1;
}

pid_t pid = fork();
if (pid < 0) {
    perror("fork");
    return 1;
} else if (pid == 0) {

    execlp("ping", "ping", "-c", "1", host, (char *)NULL);
    /* If execlp returns, there was an error */
    fprintf(stderr, "Failed to exec ping: %s\\n", strerror(errno));
    _exit(127);
} else {
    /* Parent: wait for child to finish */
    int status;
    if (waitpid(pid, &status, 0) == -1) {
        perror("waitpid");
        return 1;
    }
    if (WIFEXITED(status)) {
        int code = WEXITSTATUS(status);
        printf("ping exited with status %d\\n", code);
        return code;
    } else if (WIFSIGNALED(status)) {
        printf("ping killed by signal %d\\n", WTERMSIG(status));
        return 1;
    } else {
        printf("ping finished with unknown status\\n");
        return 1;
    }
}
}

```

fork() で子プロセスを生成し、子プロセス側で execl により ping を直接実行します。

execl はシェルを経由しないため、ホスト名に含まれるメタ文字 (; や | など) は解釈されない :

waitpid で子プロセスの終了を待ち、正常終了 / シグナルによる終了の判定とステータスの出力を行う。

3.5 テスト結果

```
=== Cmdexec tests for fix_cmdexec ===
--- input #1 (len=9) ---
PING-SHIM-ARGS: -c 1 127.0.0.1
Enter host to ping: ping exited with status 0
--- input #2 (len=13) ---
Invalid host. Allowed characters: letters, digits, '.' and '-'.
Enter host to ping: --- input #3 (len=26) ---
Invalid host. Allowed characters: letters, digits, '.' and '-'.
Enter host to ping: --- input #4 (len=28) ---
Invalid host. Allowed characters: letters, digits, '.' and '-'.
Enter host to ping: --- input #5 (len=27) ---
Invalid host. Allowed characters: letters, digits, '.' and '-'.
Enter host to ping:
=== Format tests for fix_format ===
```

Tect 1: 127.0.0.1

Enter host to ping: 127.0.0.1

PING 127.0.0.1 ... Success

Tect 2: invalid_host!

Enter host to ping: invalid_host!

Invalid host. Allowed characters: letters, digits, '.' and '-'. Refused

Tect 3: 127.0.0.1; echo INJECTED

Enter host to ping: 127.0.0.1; echo INJECTED

Invalid host. Allowed characters: letters, digits, '.' and '-'. Refused

Tect 4-5: Backtick

Enter host to ping: 127.0.0.1 `echo INJECTED`

Invalid host. Allowed characters: letters, digits, '.' and '-'. Refused

Tect 6: -xyzabc

Enter host to ping: -xyzabc

Invalid host. Allowed characters: letters, digits, '.' and '-'. Refused
(check that '-' cannot be at the beginning)

3.6 結論

本修正により、vuln_cmdexec.c における Command Injection 脆弱性は除去された。

- 入力検証により危険な文字列を事前に排除
- system() を廃止し、シェルを経由しない execlp() による実行に変更
- エラーハンドリングと多層防御を実装

残存リスクとしては、ホワイトリストのルールが緩い場合や、OS 依存の挙動（ping のオプション違いなど）に注意が必要である。

まとめ

本演習では、`vuln_overflow.c`, `vuln_format`, `vuln_cmdexec.c` の 3 種類の脆弱性を含むサンプルプログラムを解析・修正した。

これらの脆弱性はいずれも入力の検証不足+安全でない API の使用に起因する典型的な実装ミスである。今回の修正では以下の基本原則（セキュアコーディングのベストプラクティス）を適用した。

- 入力値の検証（ホワイトリスト）を行うこと
- バッファサイズや引数長を明示的に制限すること（安全な API を利用）
- シェルを介さない安全な実行方法を採用すること（必要なら権限を限定する）
- 多層防御（Defense-in-Depth）を採用し、単一の対策に依存しないこと

学んだこと・教訓

- ユーザー入力を決して信用しない — すべての入力を検証（バリデーション）すること。
- 境界付き関数を使用する — 境界チェックのない関数（`gets`, `strcpy` など）は避けること。
- 多層防御（Defense in depth）を採用する — 複数の防御層（入力検証・安全な API・権限制御など）を組み合わせること。
- コードレビューは非常に重要 — システムティックなレビューとテストを実施すること。
- 使用した参考文献・情報源の明示 — 参照した資料を記録して根拠を残すこと。

References :

- CWE (Common Weakness Enumeration): CWE-120, CWE-121, CWE-134
- OWASP Top 10: Buffer overflow, injection attacks
- CERT C Coding Standard: STR31-C, STR38-C
- ISO/IEC 9899 (C Standard):